

Organização e Arquitetura de Computadores

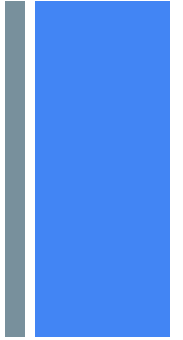
Estrutura do Computador

Prof. Msc. Lucas Reis

lucas.oliveira@puc-campinas.edu.br



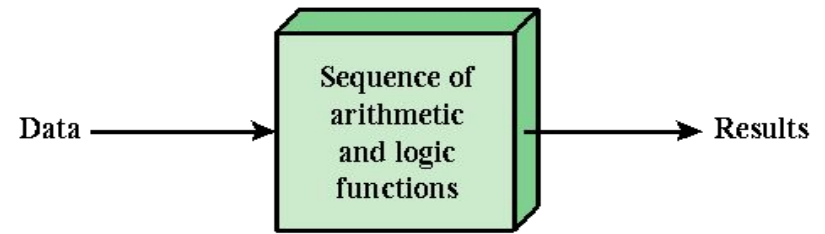
Componentes do Computador



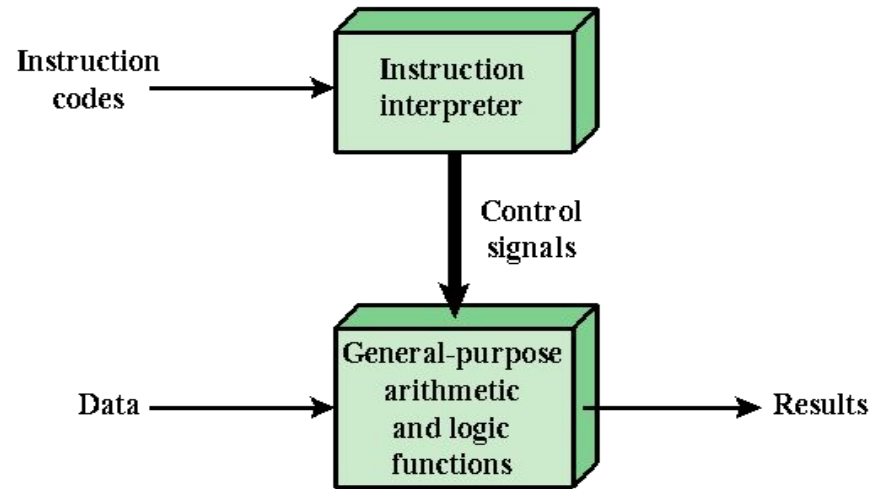
- Designs contemporâneos baseados nos conceitos apresentados por John von Neumann
- *Arquitetura de von Neumann*
 - Dados e instruções armazenados em uma memória única
 - Conteúdo dessa memória é endereçável por localização, sem se preocupar com o tipo de dado existente
 - Execução ocorre de maneira sequencial (de uma instrução para a próxima)
- *Programa hardwired*
 - Resultado da interconexão entre vários componentes na configuração desejada para executar instruções
 - Predecessor do *software*



Abordagem de Hardware e Software



(a) Programming in hardware



(b) Programming in software

Figure 3.1 Hardware and Software Approaches

Software

- Sequência de código ou instruções
- Parte do hardware interpreta as instruções e gera sinais de controle
- Inserimos uma nova sequência de códigos para cada programa no lugar de construir novo hardware

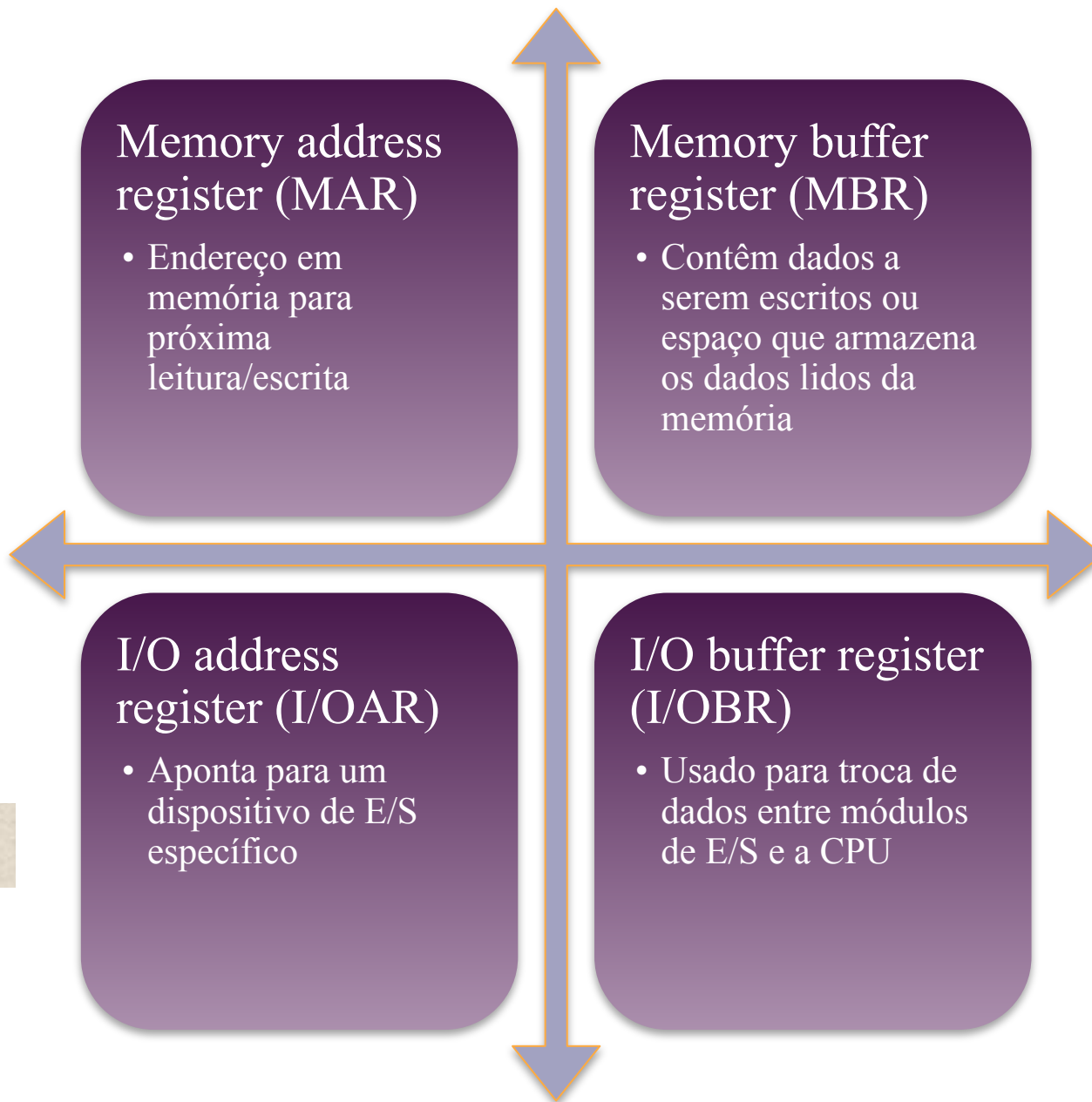
Componentes Principais:

- CPU
 - Interpreta instruções
 - Apresenta funções lógicas e aritméticas de propósito geral
- Componentes de E/S
 - Módulo de entrada
 - Contém componentes básicos para receber dados e instruções e convertê-los em sinais internos usáveis pelo sistema
 - Módulo de saída
 - Mecanismo para reportar os resultados

Software

I/O
Components





MEMÓRIA

MAR

MBR

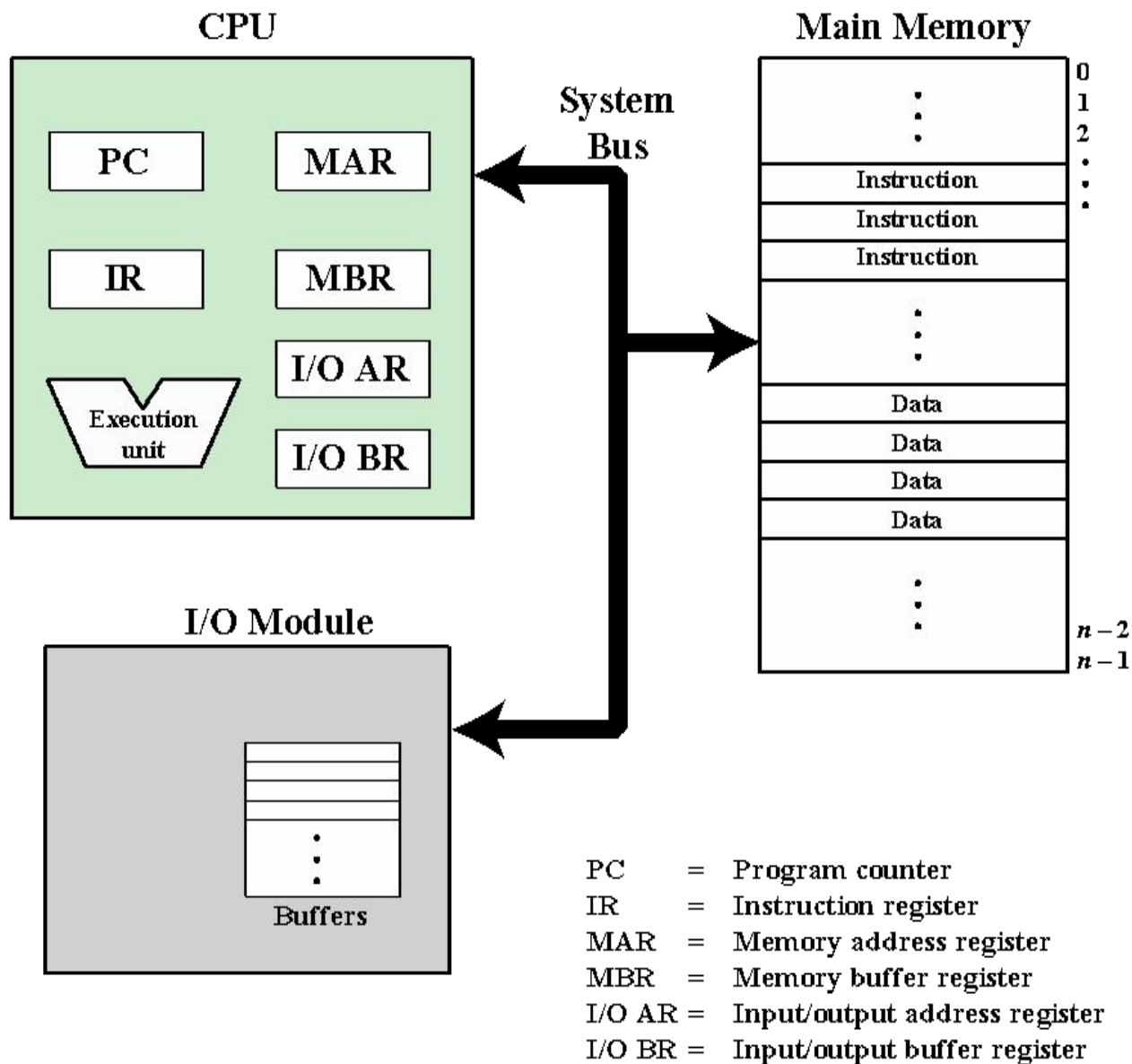


Figure 3.2 Computer Components: Top-Level View

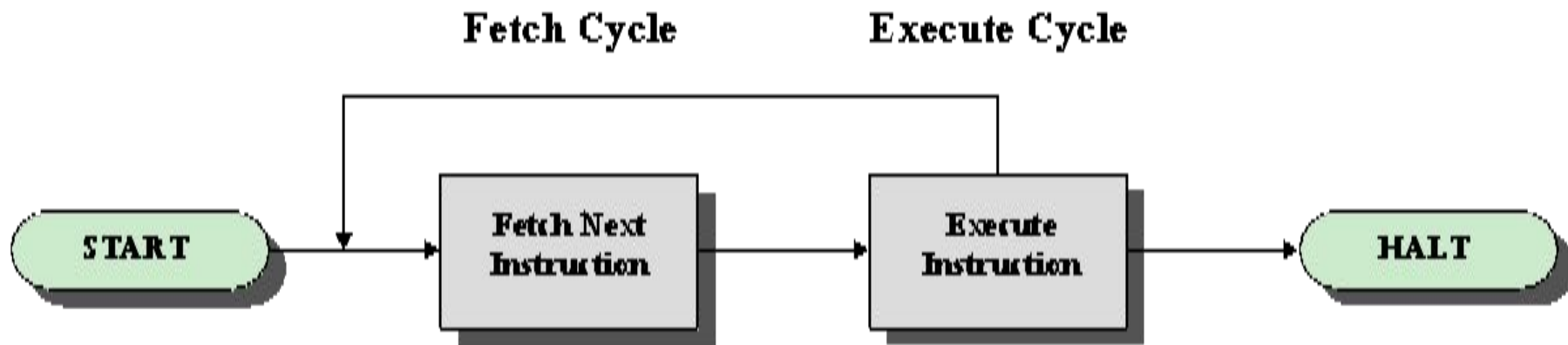
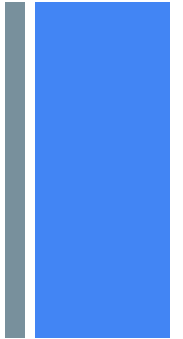


Figure 3.3 Basic Instruction Cycle



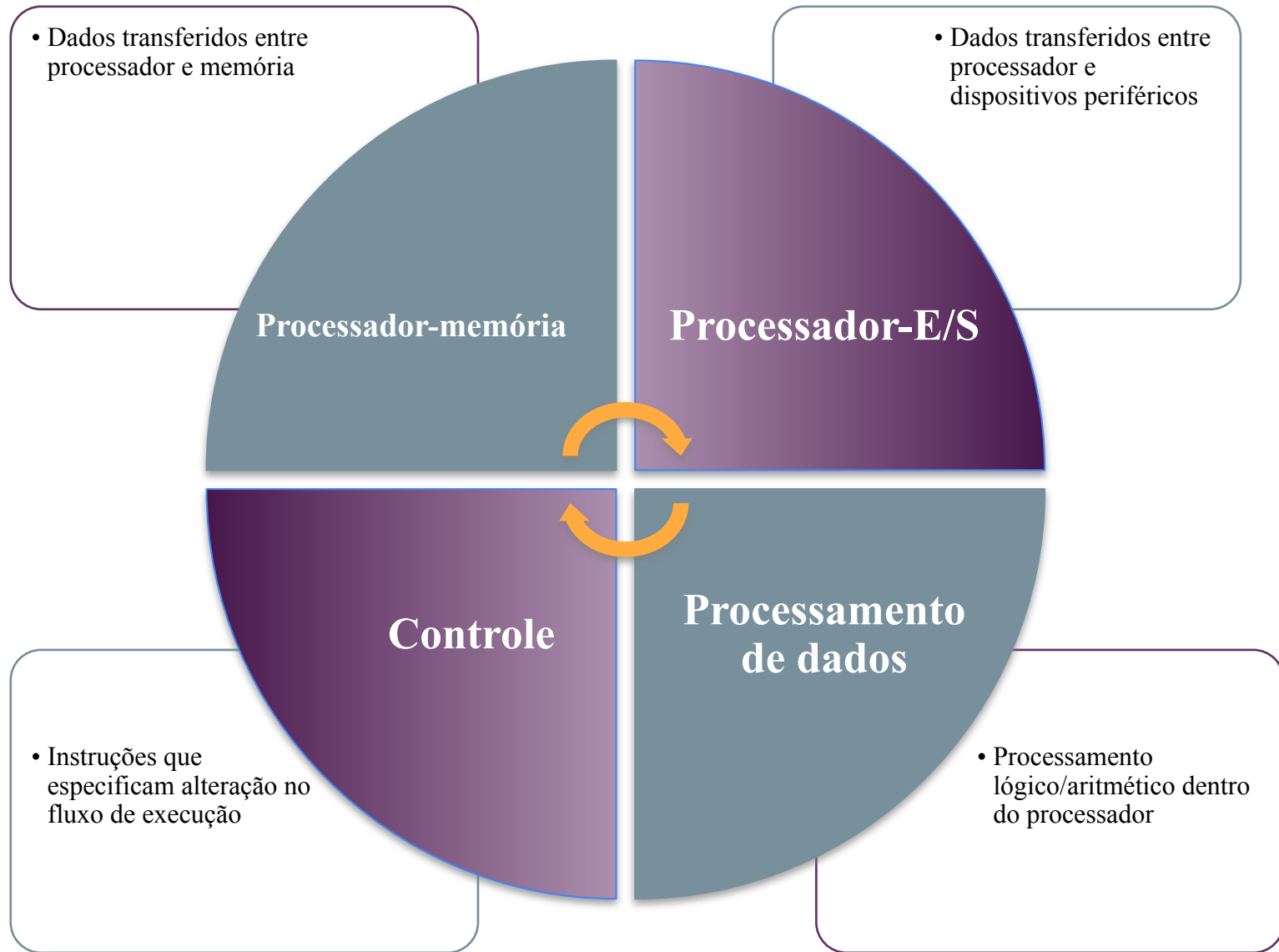
Ciclo de fetch/busca



- No início de cada ciclo de instrução, o processador busca uma instrução da memória
- O contador de programa (PC) armazena o endereço da instrução que devemos buscar depois
- O processador incrementa o PC após cada busca de instrução, para garantir que a próxima busca seja a instrução na sequência
- A instrução buscada é carregada no *instruction register* (IR)
- O processador interpreta a instrução e computa a ação requisita

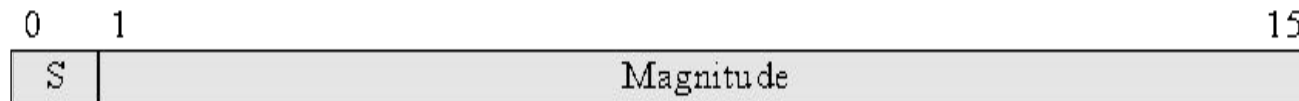


Categorias de Ações





(a) Instruction format



(b) Integer format

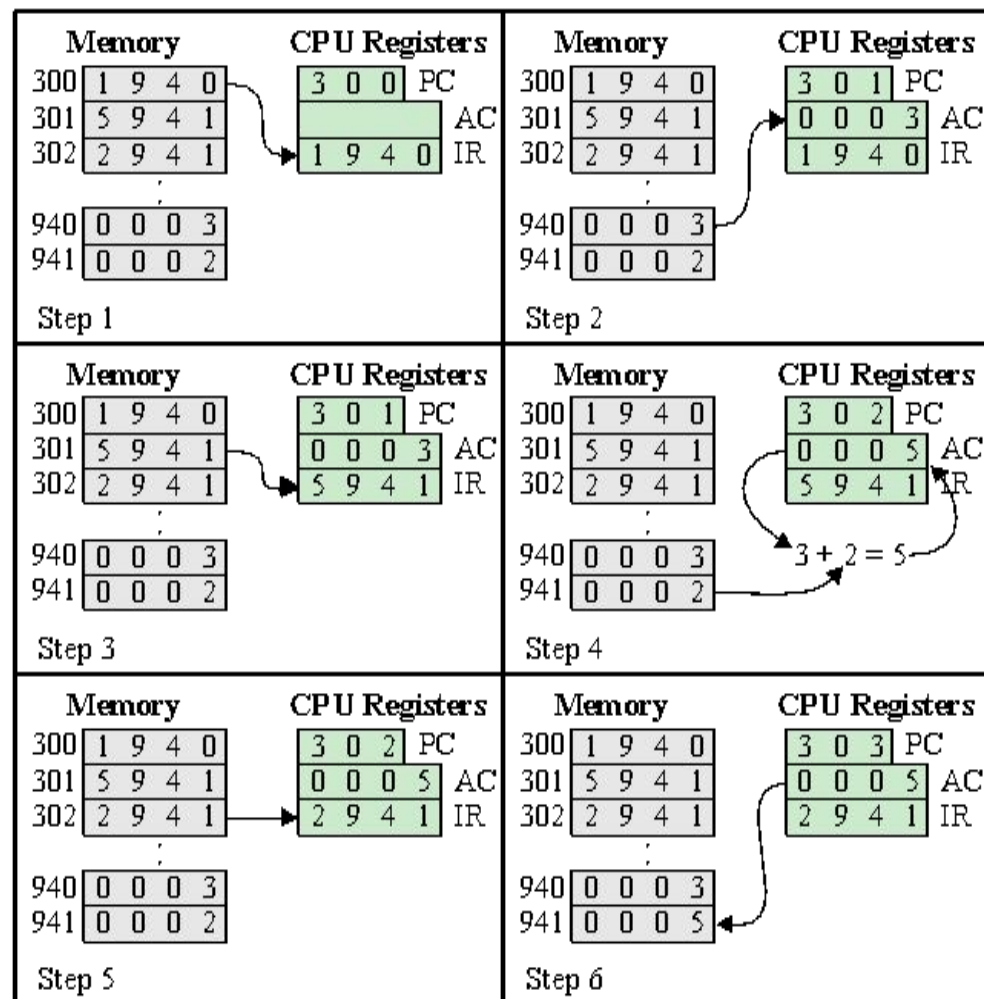
Program Counter (PC) = Address of instruction
Instruction Register (IR) = Instruction being executed
Accumulator (AC) = Temporary storage

(c) Internal CPU registers

0001 = Load AC from Memory
0010 = Store AC to Memory
0101 = Add to AC from Memory

(d) Partial list of opcodes

Figure 3.4 Characteristics of a Hypothetical Machine



**Figure 3.5 Example of Program Execution
(contents of memory and registers in hexadecimal)**

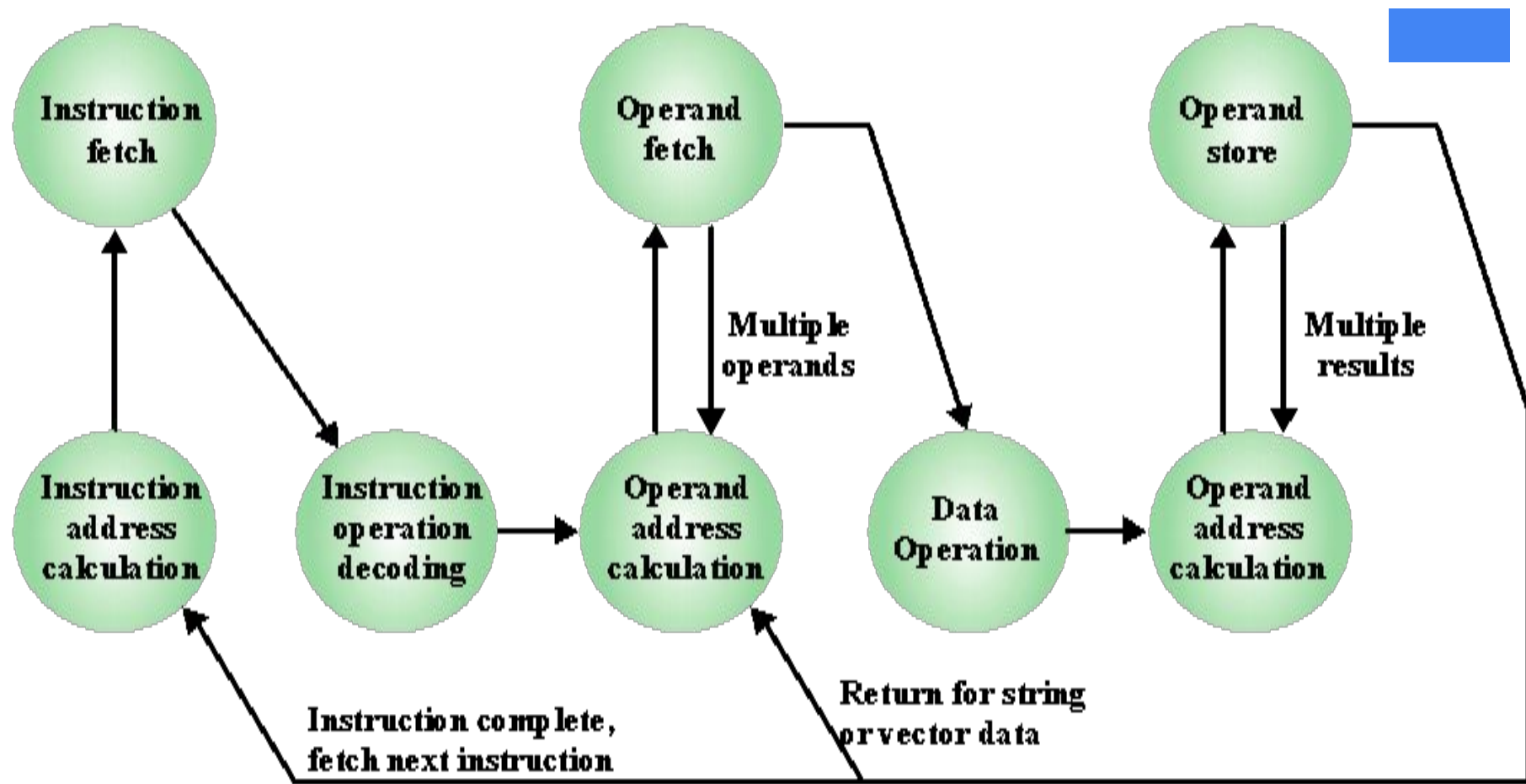


Figure 3.6 Instruction Cycle State Diagram



Program	Generated by some condition that occurs as a result of an instruction execution, such as arithmetic overflow, division by zero, attempt to execute an illegal machine instruction, or reference outside a user's allowed memory space.
Timer	Generated by a timer within the processor. This allows the operating system to perform certain functions on a regular basis.
I/O	Generated by an I/O controller, to signal normal completion of an operation, request service from the processor, or to signal a variety of error conditions.
Hardware failure	Generated by a failure such as power failure or memory parity error.

Table 3.1

Classes of Interrupts

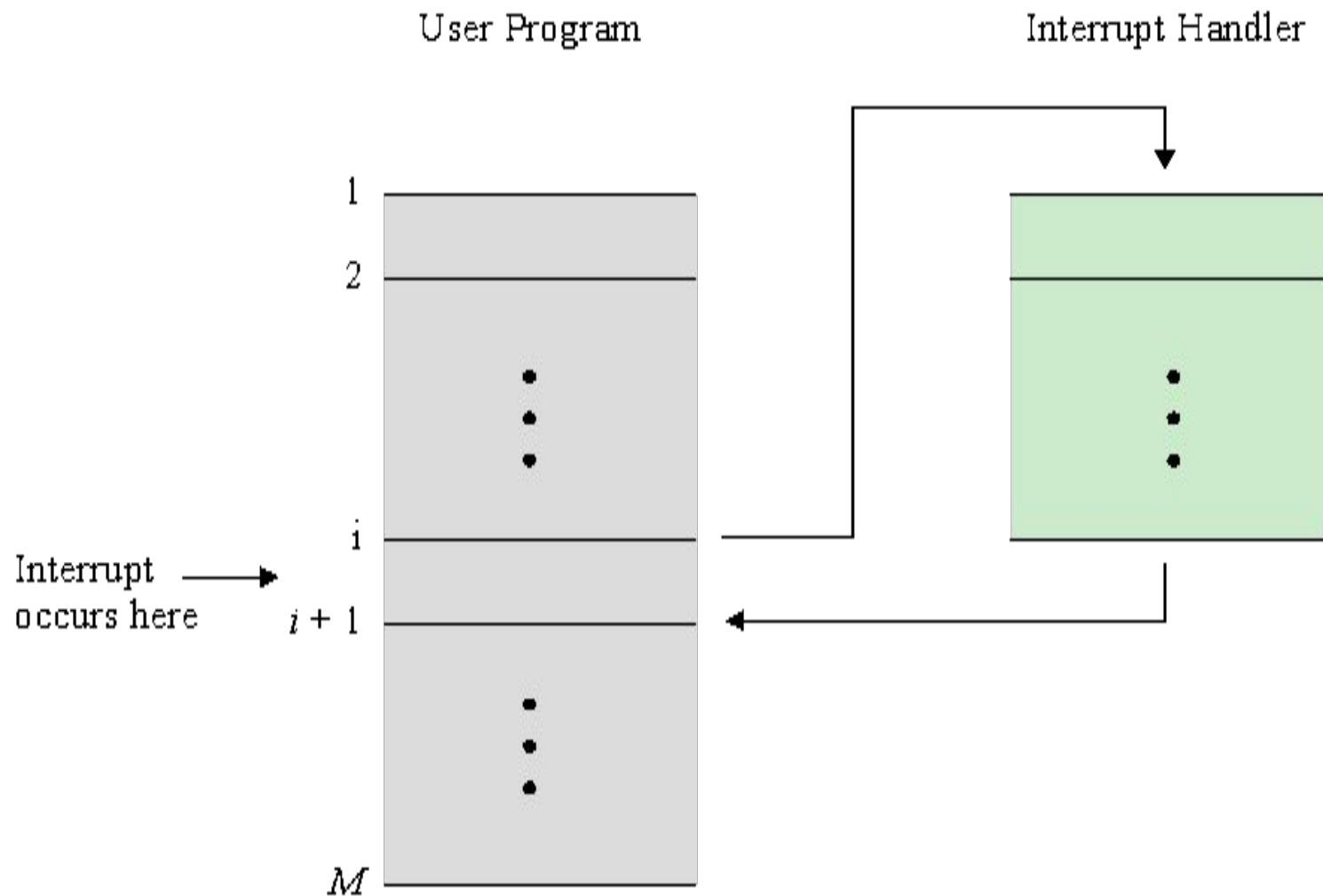


Figure 3.8 Transfer of Control via Interrupts

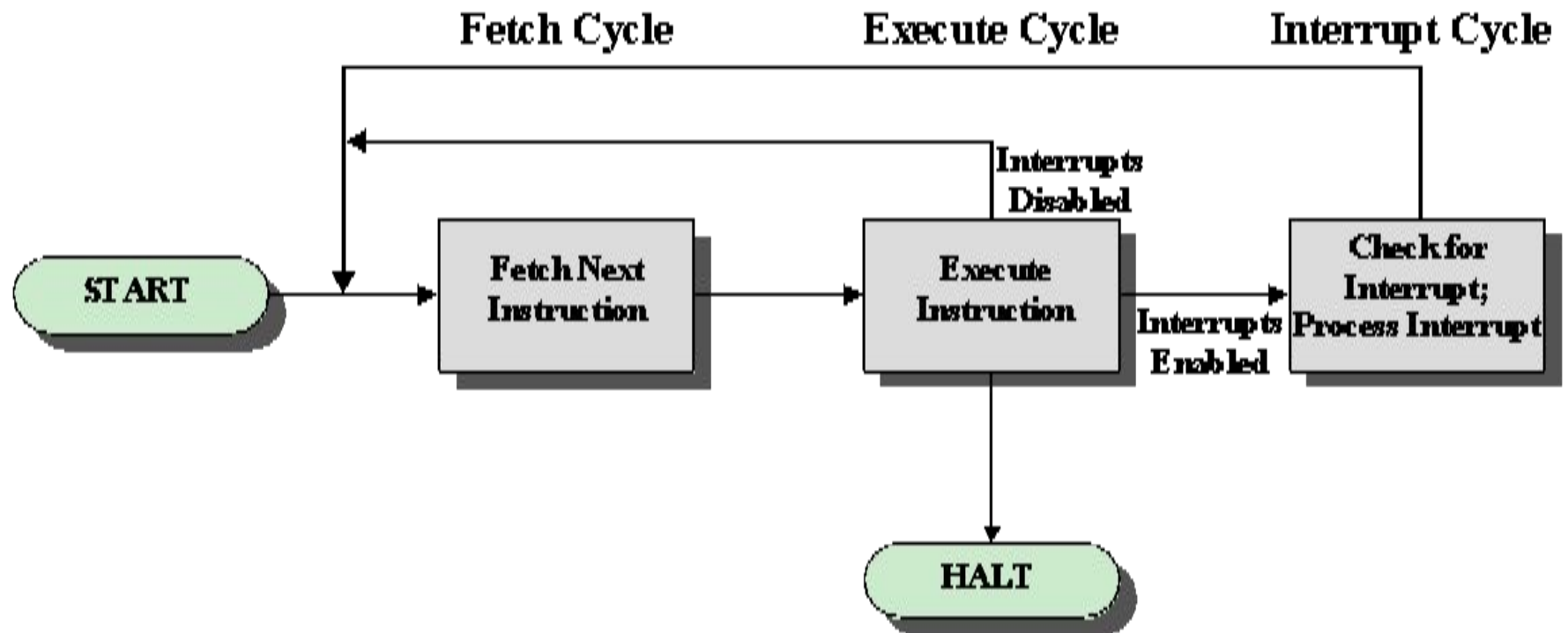


Figure 3.9 Instruction Cycle with Interrupts

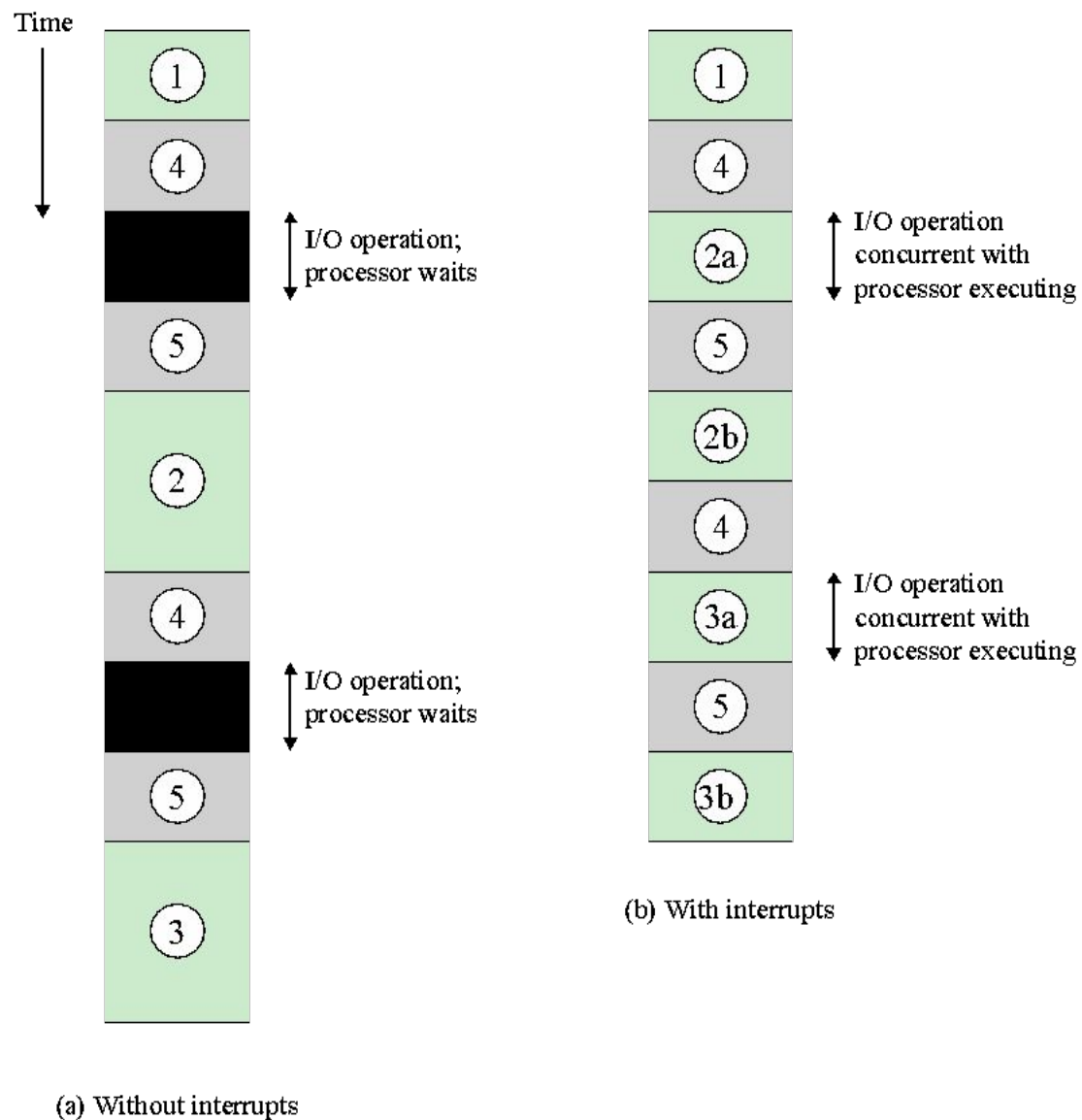
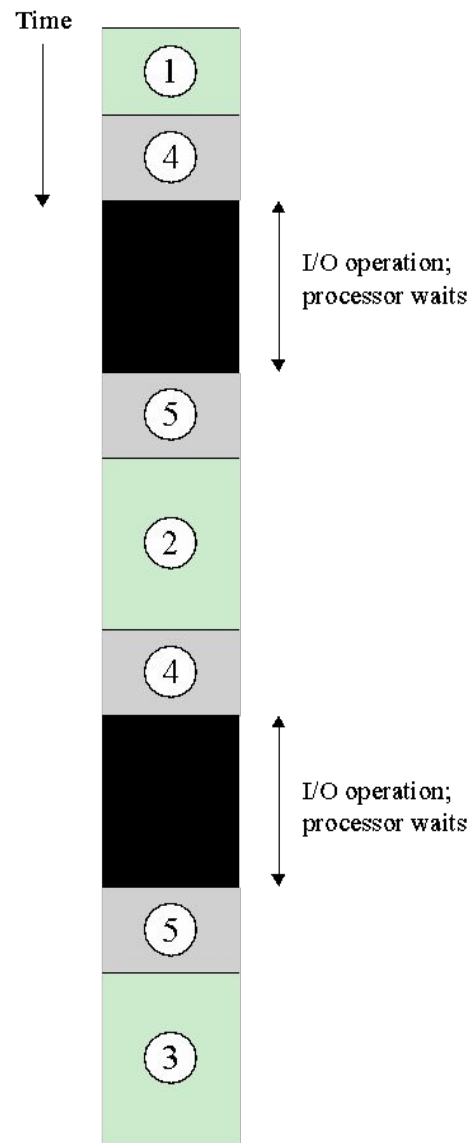
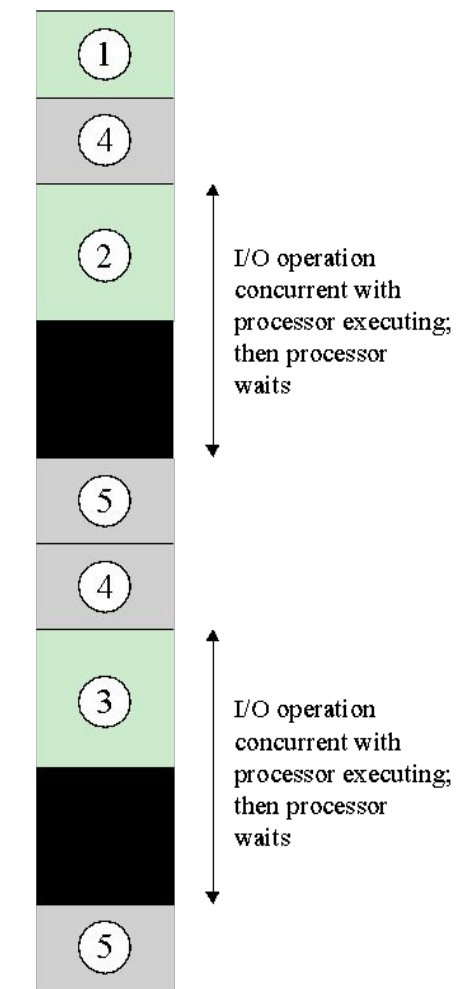


Figure 3.10 Program Timing: Short I/O Wait



(a) Without interrupts

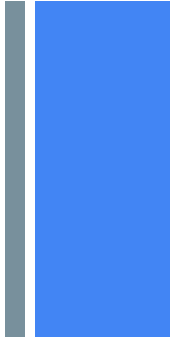


(b) With interrupts

Figure 3.11 Program Timing: Long I/O Wait



Funções de E/S



- Módulos de E/S podem trocar informações diretamente com o processador
- Processador pode ler ou escrever dados em módulo de E/S
 - Processador identifica um dispositivo específico controlado por um módulo de E/S particular
 - Referencia instruções de E/S no lugar de instruções de memória
- Módulos de E/S podem trocar dados diretamente com a memória
 - Para reduzir o tempo de CPU da transmissão de dados, é permitido ao dispositivo se conectar com a memória por intermédio do processador
 - Chamado de DMA (*Direct Memory Access*)

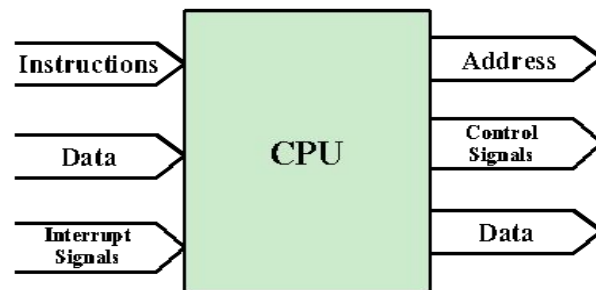
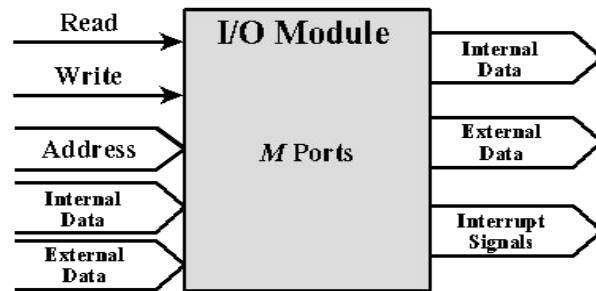
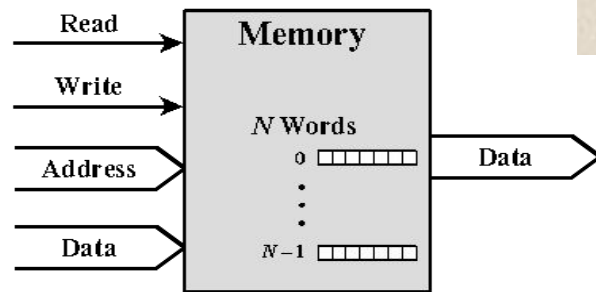
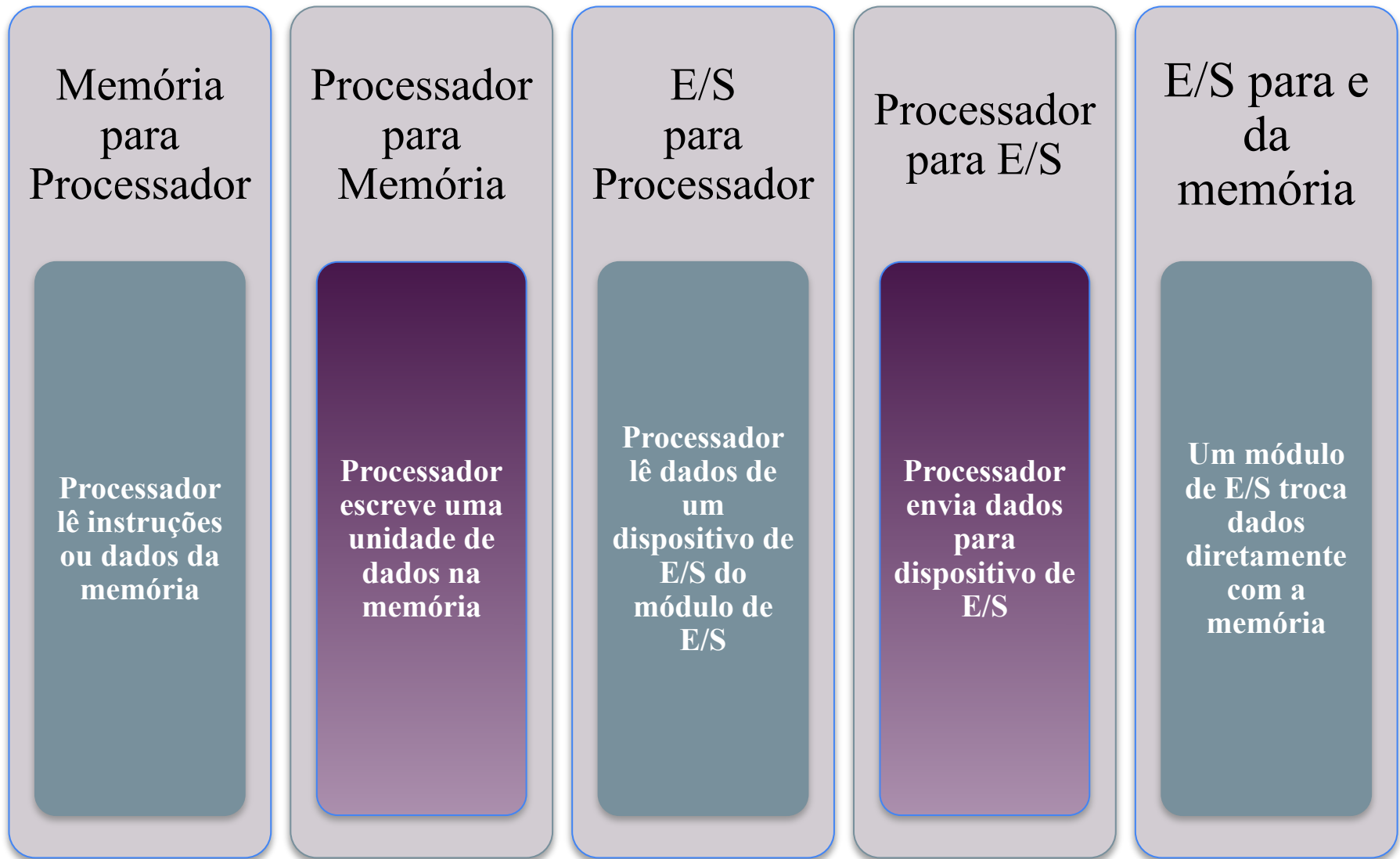
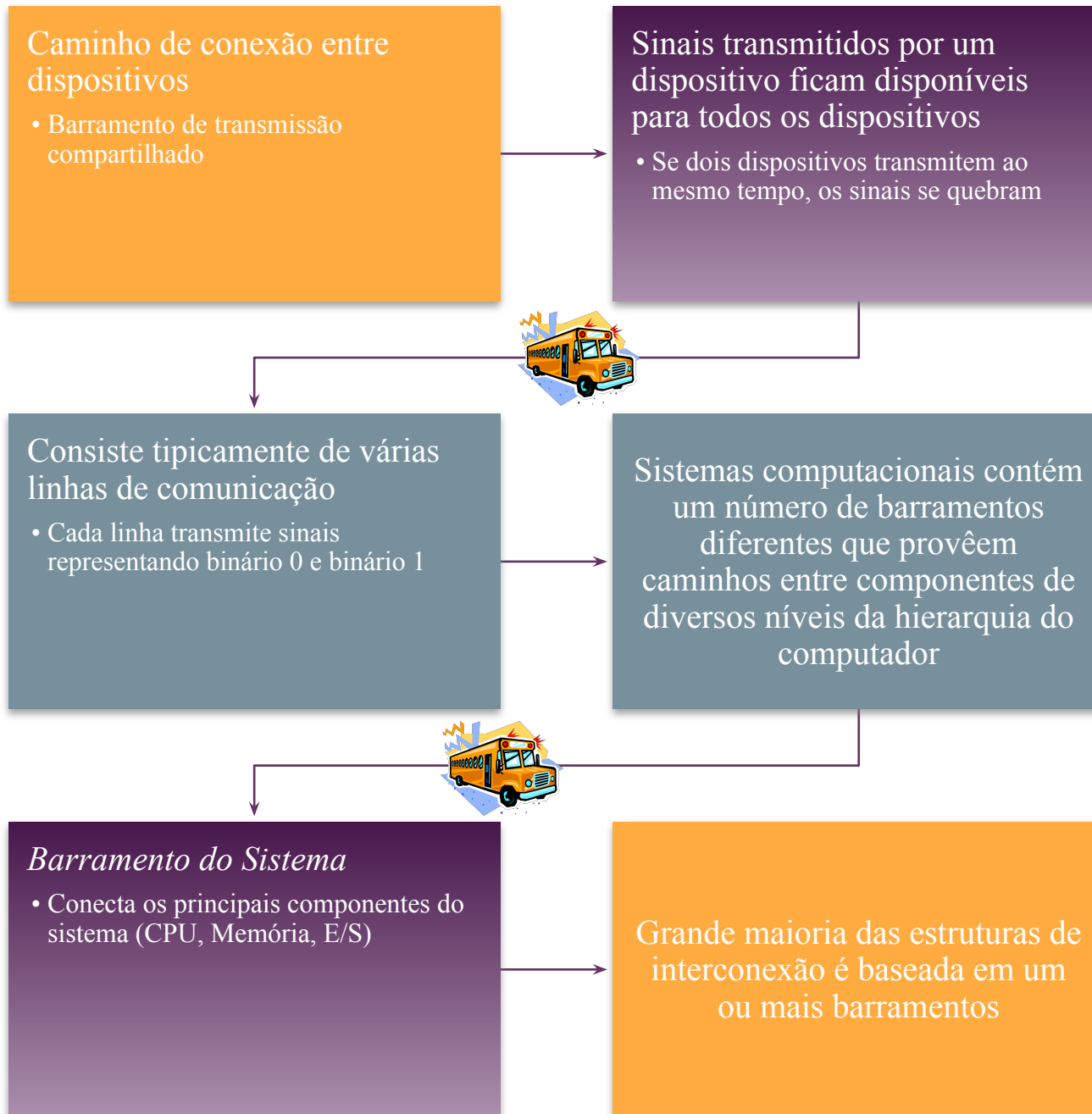


Figure 3.15 Computer Modules

Estruturas de conexão devem lidar com os seguintes tipos de transferência de dados:

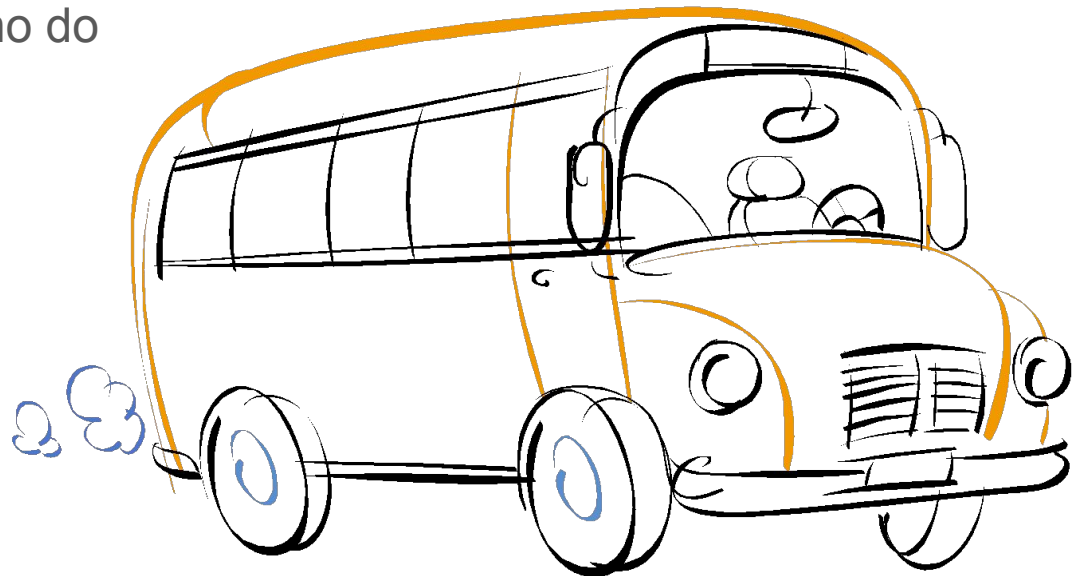


Conexões de Barramento

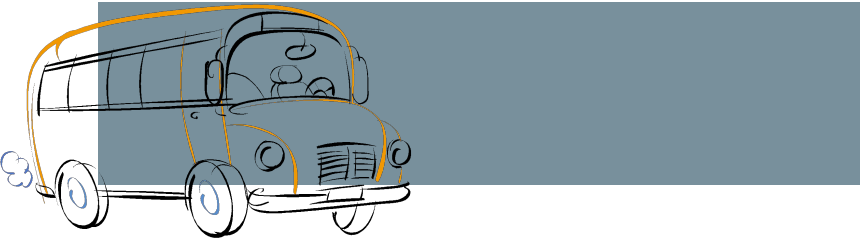


Barramento de dados

- Linhas de dados que provêm caminhos para transmissão de dados
- Consiste de 32, 64, 128, ou mais linhas
- O número de linhas equivale à largura do barramento
 - Determina quantos bits podem ser transferidos ao mesmo tempo
- *A largura*, portanto, é essencial para nivelar o desempenho do sistema

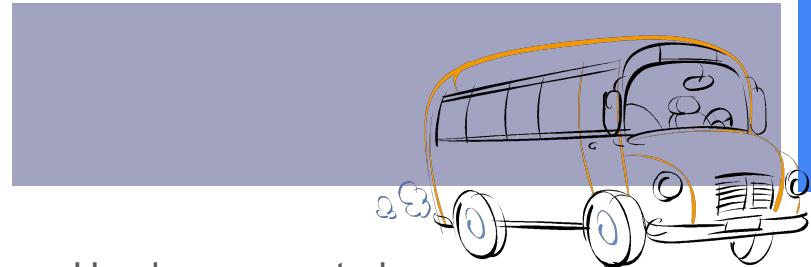


+ Barramento de Endereçamento



- Designa a fonte e destino dos dados no barramento de dados
 - Se o processador deseja ler uma palavra de dados da memória, ele coloca os endereços da palavra nas linhas de endereçamento
- Largura determina o tamanho máximo possível da capacidade de memória do sistema
- Usada para endereçar as portas de E/S
 - Bits mais significativos selecionam um módulo particular no barramento e os menos significativos selecionam a localização do elemento dentro do módulo

Barramento de Controle



- Usado para controlar o acesso do barramento
- Como os barramentos de dados e endereçamento são compartilhados, precisamos sincronizar seu uso
- Sinais de controle transmitem comandos e informações de temporização entre os módulos conectados
- Sinais de temporização indicam o prazo de validade das informações de dados e endereços
- Sinais de comando especificam operações a serem executadas

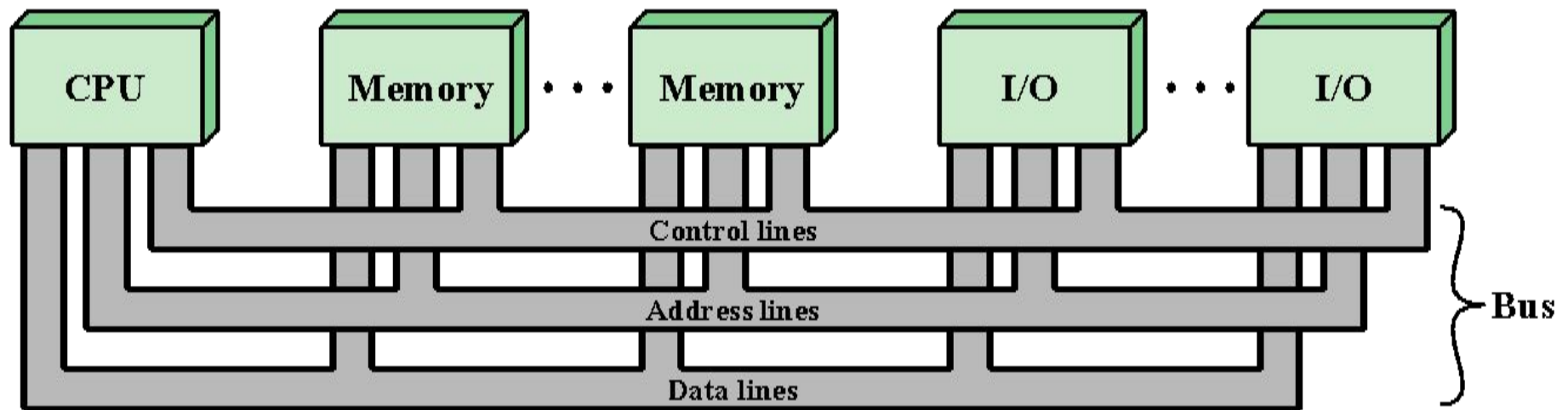
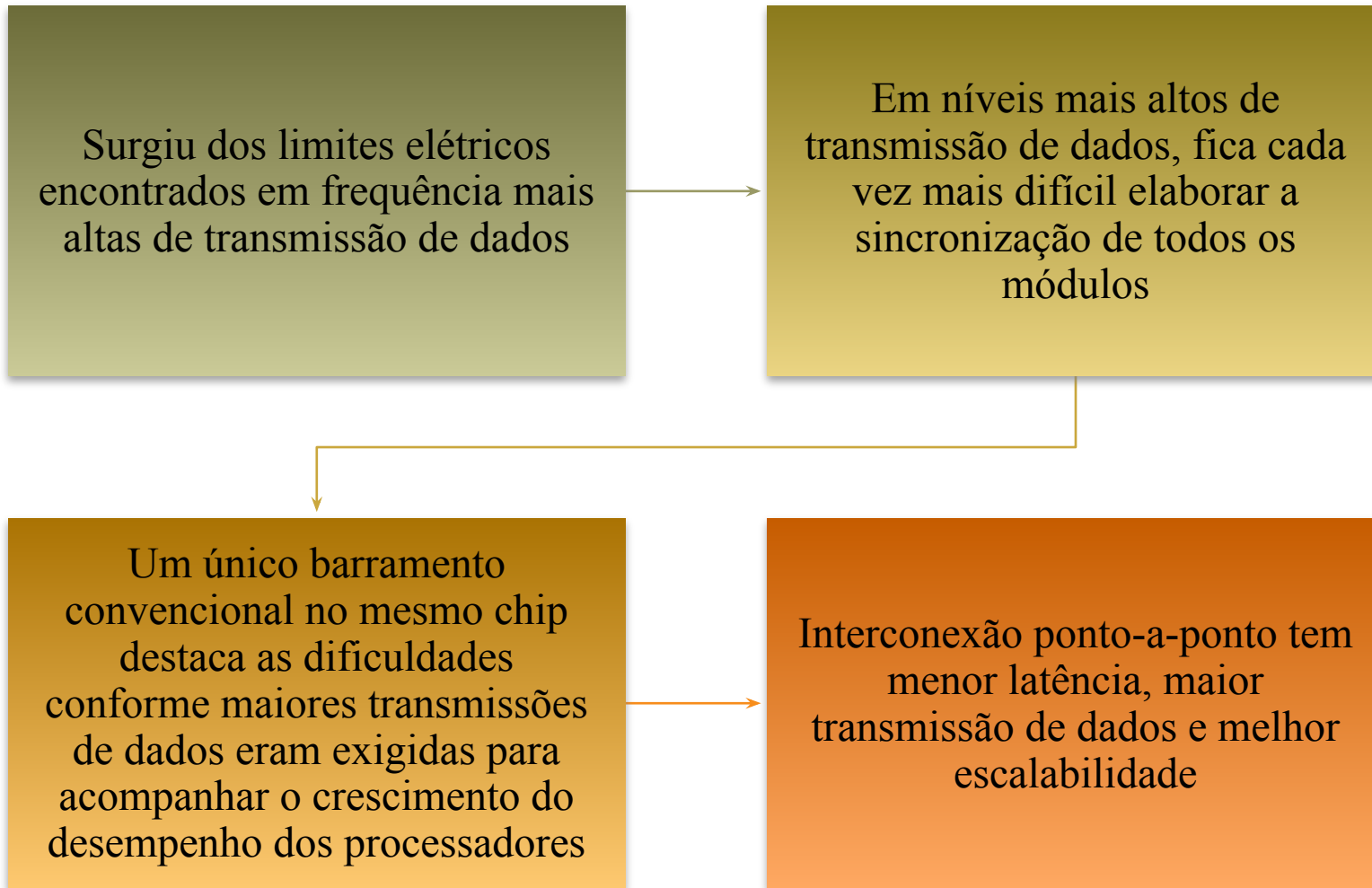
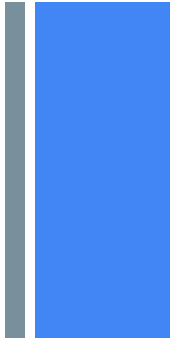


Figure 3.16 Bus Interconnection Scheme



Interconexão Ponto-a-Ponto



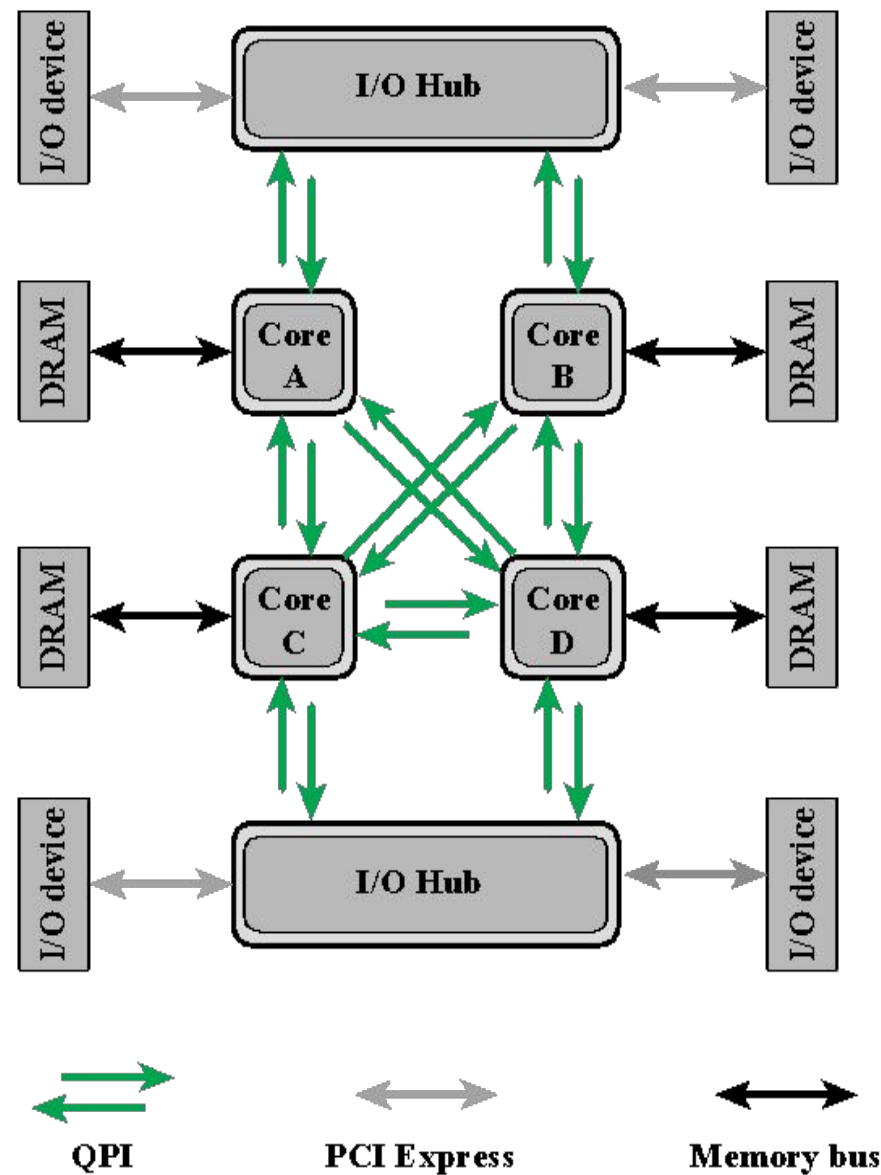


Figure 3.17 Multicore Configuration Using QPI